KPR Institute of Engineering and Technology

(Autonomous)

Avinashi Road, Arasur, Coimbatore – 641 407

Department of Artificial Intelligence and Data Science

CASE STUDY REPORT

Exploratory Data Analysis, Visualization and Deep Learning on the Breast Cancer Wisconsin (Diagnostic) Dataset

Course: U21ADP05 – Exploratory Data Analysis and Visualization
Assignment II (Case Study) | Batch 2024-2028 | Year / Sem: III / V
Course Coordinator: Ms. Revathy M

Item	Details
Student Name	Soundarya B
Roll No.	24AD111
Department / Year	AI & DS / III Year
Date of Submission	11/10/2026
GitHub Repository	https://github.com/soundaryaXai/eda-breast-cancer-dl

1. Abstract

Early and accurate classification of breast tumours as benign or malignant is important for timely treatment. This case study analyses the Breast Cancer Wisconsin (Diagnostic) dataset, which has 569 patient records and 30 numeric features computed from digitised fine-needle-aspirate images of breast masses. A complete exploratory data analysis (EDA) was carried out: data quality checks, outlier detection, winsorisation, standardisation and a stratified 70/15/15 split. Eight static visualisations and one interactive chart were produced, covering class balance, distributions, correlations, distribution fitting (Normal, Log-normal) and a Binomial model. A multi-layer perceptron (MLP) was tuned over 16 configurations and evaluated on unseen data, achieving 96.5% accuracy and an AUC of 0.990. Size-related and concavity-related features were found to be the strongest indicators of malignancy.

2. Introduction and Objective

Breast cancer is among the most common cancers worldwide, and a fine-needle aspirate (FNA) is a minimally invasive test in which cells are examined under a microscope. Measuring cell-nucleus characteristics such as size, shape and texture from the digitised image can support pathologists by providing an objective, data-driven second opinion. This is a real-world healthcare application in which exploratory analysis helps in understanding which measurements matter and a learning model helps in automating the decision.

Objectives:

1. To select a real-world dataset with at least 15 features and perform EDA and pre-processing (missing values, duplicates, outliers, scaling, splitting).
2. To create at least five meaningful visualisations covering univariate, bivariate/multivariate, continuous and discrete probability distribution analysis, with interpretation.
3. To build a deep-learning (MLP) classifier, tune basic hyper-parameters and analyse feature importance.
4. To evaluate the model using loss/accuracy curves, confusion matrix and ROC-AUC and to draw practical conclusions.

3. Dataset Description

Property	Description
Name / Source	Breast Cancer Wisconsin (Diagnostic) – UCI Machine Learning Repository (Wolberg, Street & Mangasarian); loaded via scikit-learn's load_breast_cancer()
Records	569 patients
Features	30 numeric (float64) predictors + 1 target
Target	Diagnosis: Malignant (0) or Benign (1) – 357 benign (62.7%), 212 malignant (37.3%)

Property	Description
Feature groups	10 cell-nucleus properties – radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension – each reported as the mean, the standard error and the 'worst' (mean of the three largest values), giving 10 × 3 = 30 features
Data type	Numerical (tabular)

Basic statistics of selected features (full table in outputs/summary_statistics.csv):

Feature	Mean	Std	Min	Median	Max	Skew
mean radius	14.127	3.524	6.981	13.37	28.11	0.942
mean texture	19.29	4.301	9.71	18.84	39.28	0.65
mean area	654.889	351.914	143.5	551.1	2501	1.646
mean concavity	0.089	0.08	0	0.062	0.427	1.401
worst perimeter	107.261	33.603	50.41	97.66	251.2	1.128

4. EDA and Preprocessing

4.1 Methods used

Step	What was done and result
Missing values	isna().sum() → 0 missing values; no imputation required. (Had any been found, the median would be used for skewed features.)
Duplicates	duplicated().sum() → 0 duplicate records.
Outlier detection	IQR rule (values beyond 1.5×IQR). 171 of 569 records have at least one outlying feature; the most affected are area error (65), radius error (38), perimeter error (38) and worst area (35).
Outlier handling	Winsorisation at the 1st and 99th percentiles, with limits computed on the training set only. Records were not deleted because extreme values in tumour size are clinically genuine, and removing them would delete malignant cases.
Skewness	22 of 30 features have |skewness| > 1 (right-skewed), mainly size and error features.
Encoding	Target already numeric (0/1); a text label column ('diagnosis') was added only for plotting. No categorical predictors, so no one-hot encoding was needed.
Scaling	Z-score standardisation (StandardScaler) fitted on the training set only, to avoid data leakage; required because features range from about 0.05 (fractal dimension) to over 2500 (area).
Splitting	Stratified split: train 398 (70%), validation 85 (15%), test 86 (15%), random_state = 42; class ratio preserved in each part.

4.2 Insights from EDA

- The data is clean (no missing values/duplicates) and moderately imbalanced (62.7% benign vs 37.3% malignant), so accuracy alone is not enough and recall/AUC are also reported.
- Heavy multicollinearity exists: 15 feature pairs have $|r| > 0.95$, e.g. mean radius vs mean perimeter ($r = 0.998$), because perimeter and area are geometric functions of radius.
- Size/shape features (worst concave points $r = -0.794$, worst perimeter $r = -0.783$) correlate strongly with the diagnosis, whereas texture error, fractal dimension and smoothness error are almost uncorrelated with it ($|r| < 0.07$).

5. Data Visualization

Visualisations were produced in Python (Matplotlib, Seaborn, SciPy, Plotly). They follow the seven stages of data visualisation (Ben Fry): the mapping is given in Table 5.1.

Stage	Activity in this study	Where
1. Acquire	Loaded the UCI Breast Cancer Wisconsin data	Section 3
2. Parse	Structured into a DataFrame, data types verified, target relabelled	Section 3, 4
3. Filter	Removed nothing; winsorised extreme values and selected relevant features for each chart	Section 4
4. Mine	Summary statistics, correlations, skewness, KS tests, distribution fitting	Section 4, 5
5. Represent	Chose chart types by data type: bar, histogram/KDE, box, heatmap, scatter, pairplot, Q-Q, PMF	Figures 1–8
6. Refine	Consistent colour (green = benign, red = malignant), titles, axis labels, legends, annotations	All figures
7. Interact	Interactive Plotly scatter with hover details and bubble size	outputs/interactive_scatter.html

Table 5.1: Seven stages of visualisation applied.

5.1 Class distribution (bar and pie chart)

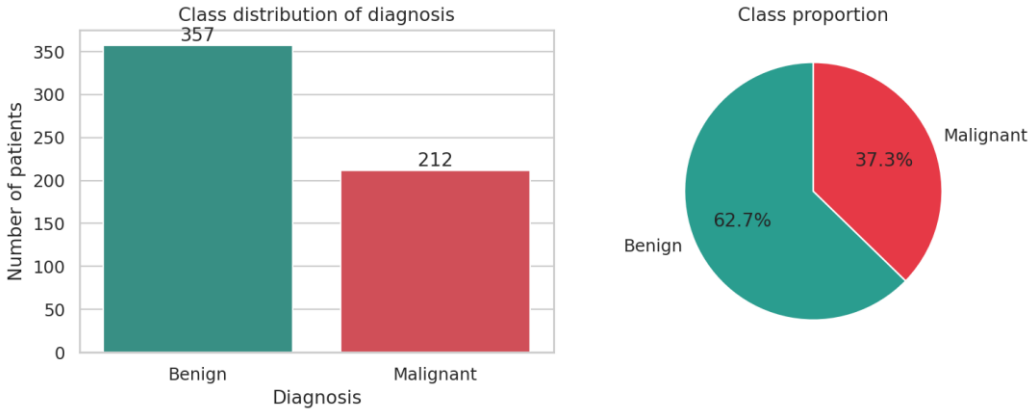


Figure 1: Class distribution of diagnosis

What it shows: the number and share of benign and malignant patients.

Why it was created: to check class balance before modelling, because imbalance affects metrics and training.

Insights: There are 357 benign (62.7%) and 212 malignant (37.3%) cases – mildly imbalanced, so stratified splitting and recall/AUC are used.

5.2 Histograms with KDE by diagnosis (univariate)

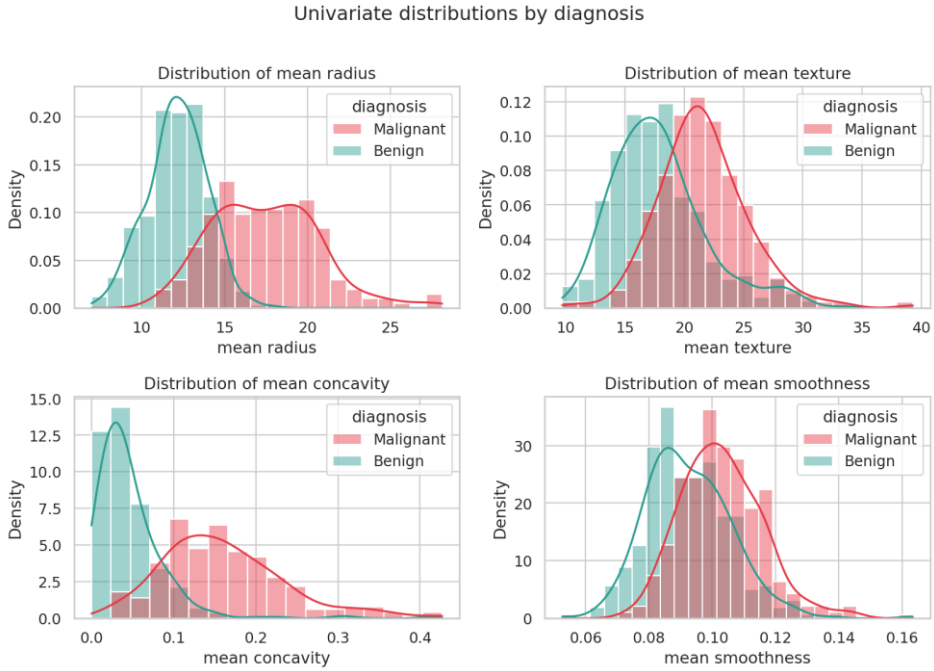


Figure 2: Histogram and density of four mean features by diagnosis

What it shows: the shape, centre, spread and skew of mean radius, texture, concavity and smoothness for each class.

Why it was created: to understand each variable individually and see which ones separate the classes.

Insights: Mean radius and mean concavity show clearly separated peaks – malignant tumours are larger with more concave contours. Mean smoothness overlaps heavily, so it is a weak discriminator. Several distributions are right-skewed.

5.3 Boxplots by diagnosis

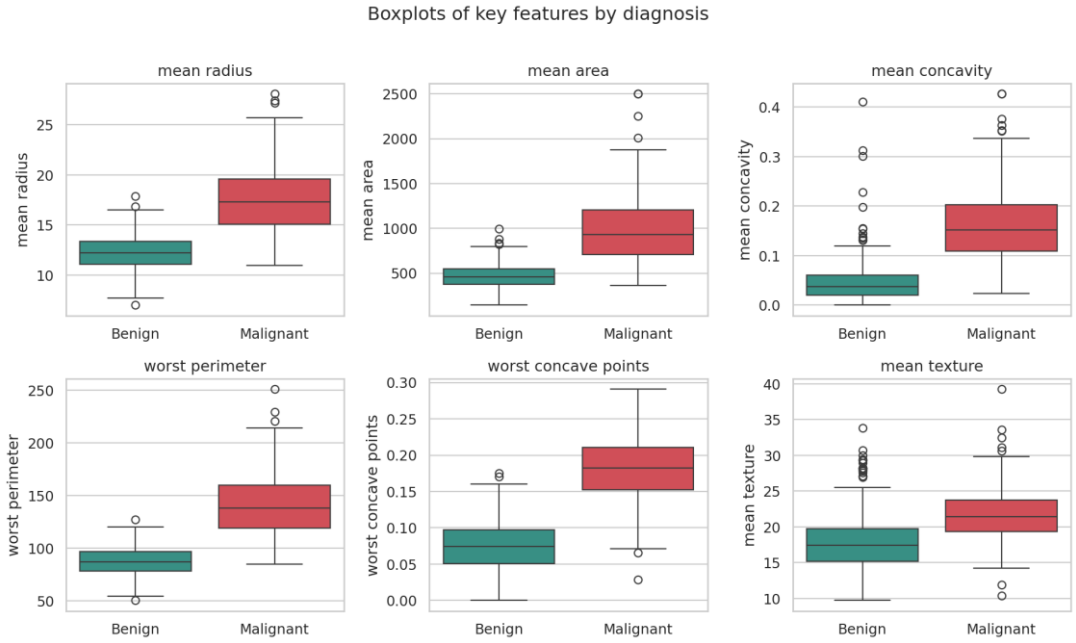


Figure 3: Boxplots of six key features by diagnosis

What it shows: median, inter-quartile range and outliers of six features for benign and malignant cases.

Why it was created: to compare class medians and visualise outliers identified by the IQR rule.

Insights: Medians of radius, area, concavity, worst perimeter and worst concave points are much higher for malignant tumours with little overlap of the boxes; mean texture shows a smaller shift. Malignant cases have more extreme high-side outliers (e.g., area).

5.4 Correlation heatmap (multivariate)

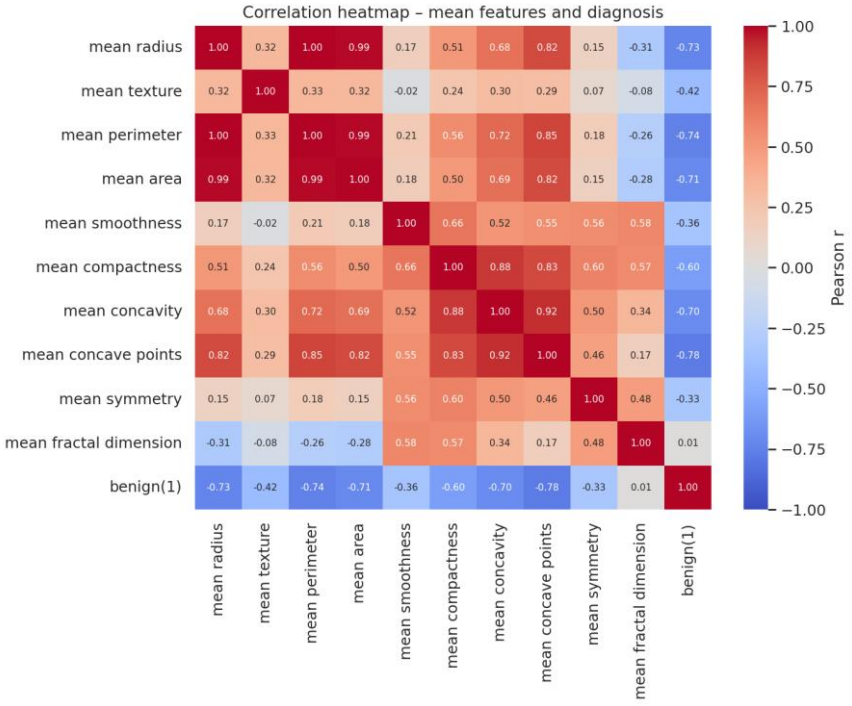


Figure 4: Pearson correlation heatmap of mean features and diagnosis

What it shows: pairwise Pearson correlation coefficients between the ten mean features and the diagnosis (benign = 1).

Why it was created: to detect redundant features and relationships with the target.

Insights: Radius, perimeter and area are almost perfectly correlated ($r \approx 0.99–1.00$); concavity, compactness and concave points form a second correlated group (r up to 0.92). All size and shape features have negative correlation with 'benign' ($r \approx -0.70$ to -0.78), while fractal dimension has none ($r = 0.01$). The redundancy suggests dimensionality reduction or regularisation could help.

5.5 Scatter plot (bivariate)

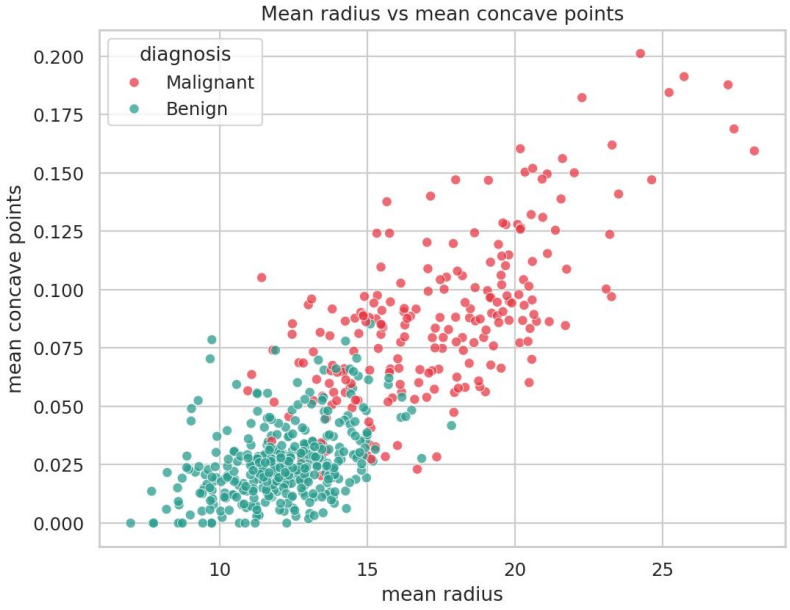


Figure 5: Mean radius vs mean concave points

What it shows: the relation between two key features with points coloured by class.

Why it was created: to see whether a combination of two features separates the classes.

Insights: The two features have r = 0.823; malignant points lie at the upper-right and benign points at the lower-left with a narrow overlap zone. This shows that the classes are almost separable even in two dimensions.

5.6 Pairplot

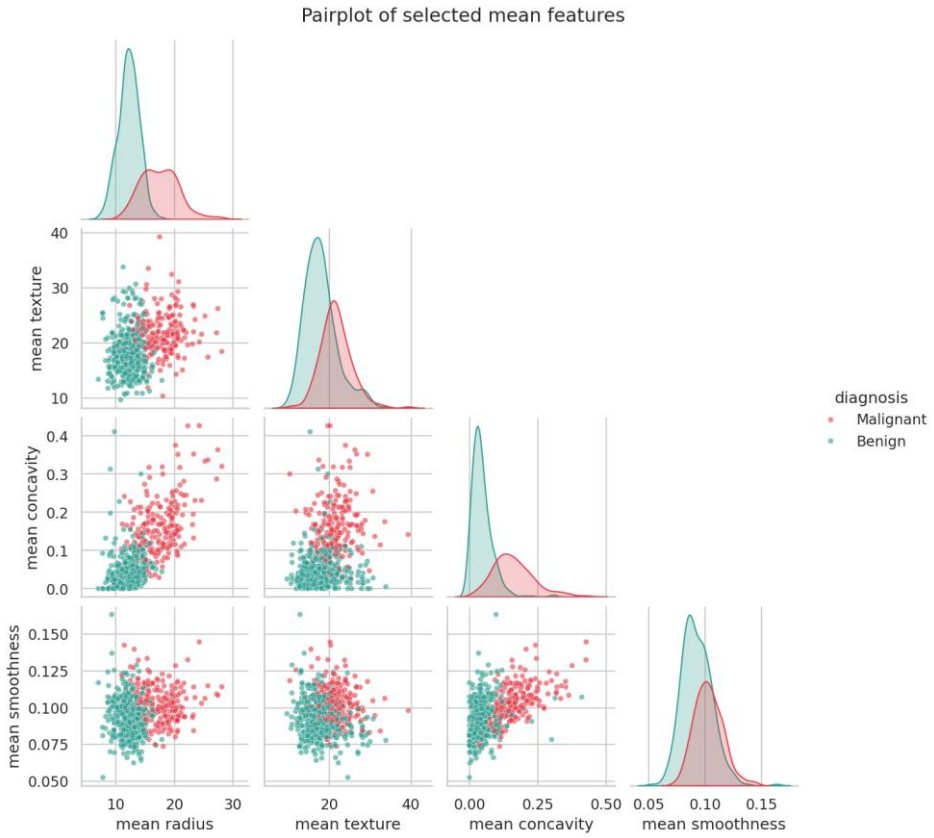


Figure 6: Pairplot of mean radius, texture, concavity and smoothness

What it shows: all pairwise scatter plots with marginal class-wise densities.

Why it was created: to explore multivariate structure at a glance.

Insights: Radius–concavity pairs give the best class separation, while smoothness–texture pairs overlap strongly. The relation between radius and concavity is positive and roughly non-linear (curved), suggesting that a non-linear model such as an MLP is suitable.

5.7 Continuous distribution fitting and Q-Q plots

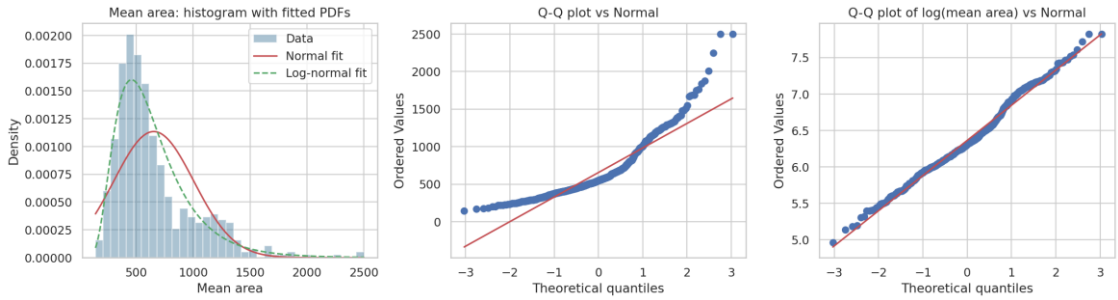


Figure 7: Normal and Log-normal fits for mean area, with Q-Q plots

What it shows: the histogram of mean area with fitted Normal and Log-normal probability density functions and the Q-Q plots against the Normal distribution before and after a log transform.

Why it was created: to identify which continuous probability distribution describes a skewed variable and to justify transformation.

Insights: Mean area is right-skewed (skewness = 1.641). The Normal fit is rejected strongly (Kolmogorov–Smirnov D = 0.156, p ≈ 1e-12), and the Q-Q plot bends upward at the tail. The Log-normal fit is much closer (D = 0.063), and log(mean area) follows a straight line in the Q-Q plot. A formal KS test still gives p = 0.02, so the fit is approximate rather than perfect. A log transform is therefore advisable for area-type variables.

5.8 Discrete distribution – Binomial model

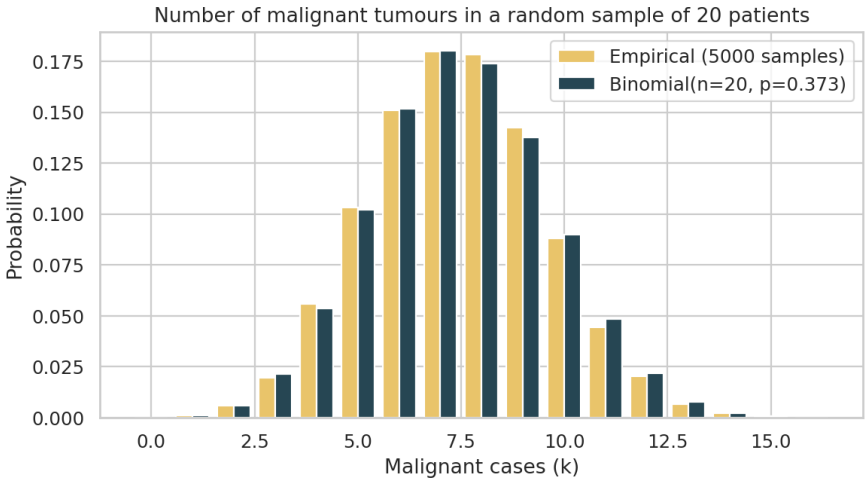


Figure 8: Binomial model for malignant cases in samples of 20 patients

What it shows: the probability of observing k malignant tumours in a random sample of n = 20 patients, empirical (5000 random samples) and the theoretical Binomial(20, p).

Why it was created: to model a discrete count variable. Each patient is a Bernoulli trial with p(malignant) = 0.3726.

Insights: Theory predicts a mean of 7.45 and standard deviation of 2.16; the simulated mean is 7.43. The empirical bars match the PMF closely, so a hospital could expect around 7 malignant cases in any group of 20 similar patients, and 3 or 4 cases above or below this are quite normal.

5.9 Interactive visualisation

An interactive Plotly bubble chart (outputs/interactive_scatter.html) shows mean radius against mean concave points, with bubble size proportional to mean area and hover tooltips displaying texture and smoothness. It fulfils the Interact stage by letting a user zoom, filter by class (legend click) and inspect individual patients.

6. Deep Learning Model

6.1 Choice of model

The data is tabular and numeric, so a feed-forward Multi-Layer Perceptron (MLP) is appropriate. It can learn the non-linear class boundary seen in the pairplot. It was implemented with scikit-learn's MLPClassifier and trained epoch by epoch (partial_fit) so that loss and accuracy per epoch could be recorded.

6.2 Architecture and training parameters

Component	Setting
Input layer	30 standardised features
Hidden layers	(16,) – ReLU activation (selected by tuning)
Output layer	1 unit, logistic (sigmoid) – probability of benign
Trainable parameters	513
Loss function	Binary cross-entropy (log loss)
Optimiser	Adam, learning rate 0.01, batch size 32
Regularisation	L2 penalty alpha = 0.01
Epochs	100 (final model)
Random seed	42

6.3 Feature analysis and importance

Feature analysis in the EDA (Section 4–5) showed correlation of each feature with the target and strong redundancy among size features. Model-based importance was measured with permutation importance on the validation set (30 repeats): the drop in accuracy when a feature is shuffled. The most influential features were worst smoothness, radius error, worst texture, area error and worst radius (Figure 13). Because the 30 features are highly correlated, importance is shared among them, and the validation set is small (85 samples), so the individual values are small and should be read as a ranking rather than an exact measure.

6.4 Hyper-parameter tuning

A basic grid search was run over hidden-layer structure {(16), (32,16), (64,32), (64,32,16)}, L2 alpha {1e-4, 1e-2} and learning rate {1e-3, 1e-2} (16 configurations, 60 epochs each). Selection used the validation loss only; the test set was never used for tuning. The five best configurations are listed below.

Hidden layers	Alpha	Learning rate	Validation accuracy	Validation loss
(16,)	0.01	0.01	0.9882	0.0259
(16,)	0.0001	0.01	0.9765	0.0322
(32, 16)	0.01	0.001	0.9765	0.0386
(32, 16)	0.0001	0.001	0.9765	0.0398
(64, 32)	0.01	0.001	0.9882	0.0398

The best configuration is a single hidden layer of 16 neurons. Deeper networks did not give a better validation loss: for a small dataset of 398 training samples, the simpler network generalises just as well, and extra depth adds only overfitting risk.

7. Result Visualization and Interpretation

7.1 Loss vs epoch

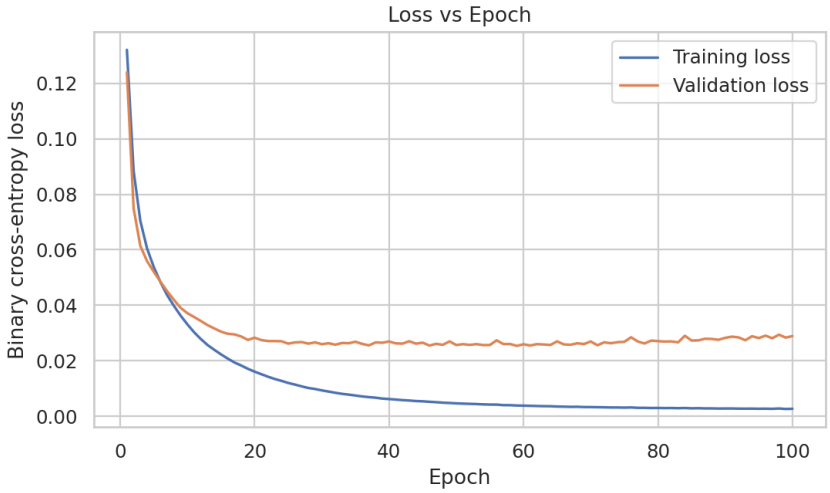


Figure 9: Training and validation loss vs epoch

What it shows: the binary cross-entropy on training and validation sets after each epoch.

Why it was created: to monitor convergence and overfitting.

Insights: Both curves fall quickly in the first 20 epochs. Training loss keeps decreasing to 0.0026, while validation loss flattens at about 0.026–0.029 (minimum at epoch 59) and rises slightly afterwards. This small gap indicates mild overfitting; early stopping at around epoch 40–60 would be sensible.

7.2 Accuracy vs epoch

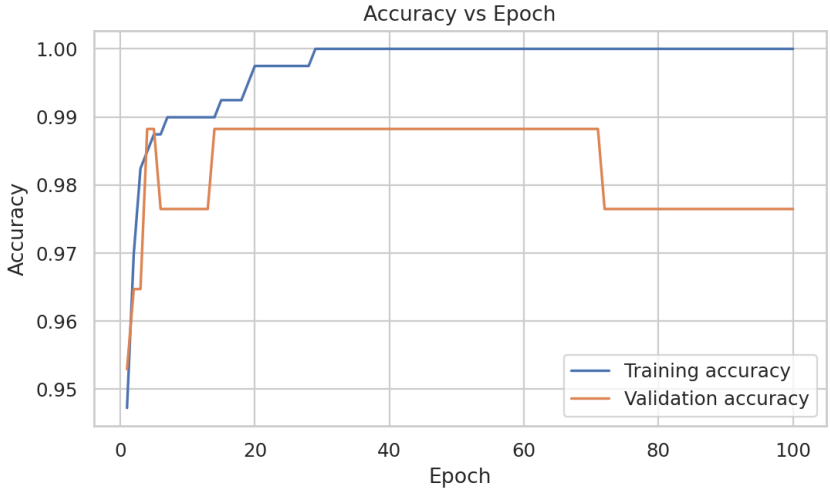


Figure 10: Training and validation accuracy vs epoch

What it shows: accuracy on both sets across the epochs.

Why it was created: to confirm that the model learns and generalises.

Insights: Training accuracy reaches 100.0% while validation accuracy stays at about 97.7%. The stable validation accuracy shows that the model generalises well to unseen patients despite the small gap.

7.3 Confusion matrix

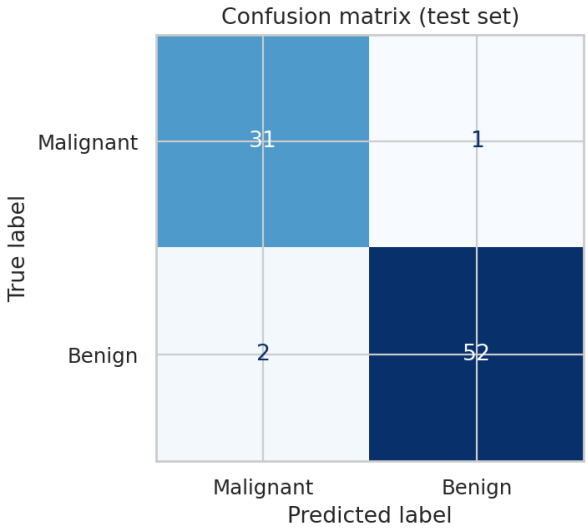


Figure 11: Confusion matrix on the test set

What it shows: counts of correct and incorrect predictions on the 86 test patients.

Why it was created: to see the types of errors, which carry different clinical costs.

Insights: Of 86 test patients, 31 malignant and 52 benign cases were classified correctly. There was 1 malignant tumour wrongly predicted as benign (false negative for cancer) and 2 benign tumours predicted as malignant (false alarms). The false negative is the more serious error clinically, so the decision threshold could be lowered to increase malignant recall.

7.4 ROC curve and AUC

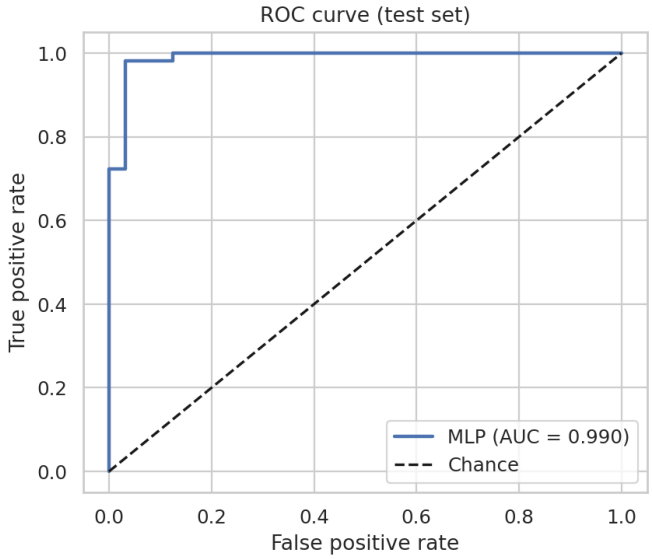


Figure 12: ROC curve on the test set

What it shows: the trade-off between true-positive and false-positive rates across all decision thresholds.

Why it was created: to evaluate class separation independent of a single threshold.

Insights: AUC = 0.990, very close to 1 (random guessing = 0.5). The curve rises sharply to the top-left, so the model ranks nearly every benign tumour above every malignant one.

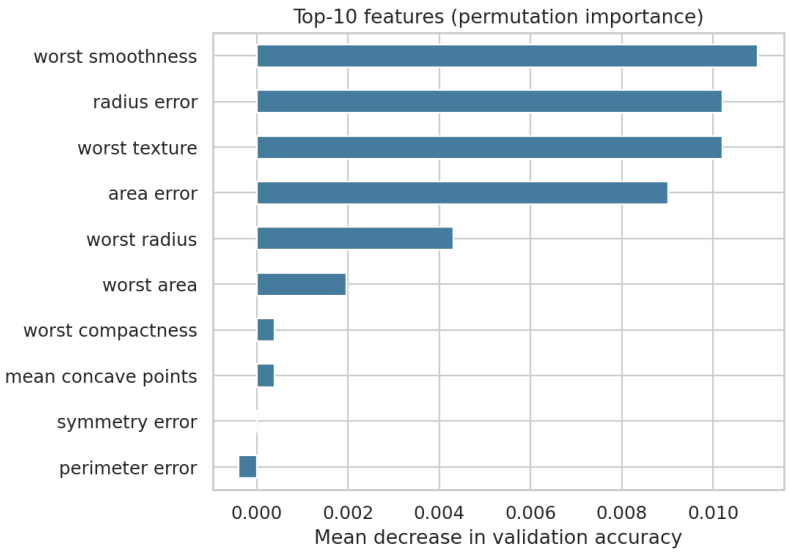


Figure 13: Permutation feature importance (validation set)

7.5 Summary of test-set performance

Metric	Value	Comment
Accuracy	96.5%	83 of 86 correct
Precision (benign)	98.1%	Few false alarms
Recall (benign)	96.3%	
F1-score (benign)	97.2%	Balance of precision and recall
ROC-AUC	0.9896	Excellent discrimination
Test log-loss	0.151	Higher than validation loss: a few confident mistakes

Note: the test set has only 86 patients, so a single misclassification changes accuracy by about 1.2 percentage points. The results should be read with this uncertainty and ideally confirmed through k-fold cross-validation.

8. Conclusion and Future Scope

8.1 Conclusion

- What was learned: the dataset is clean but contains skewed variables, outliers and strong multicollinearity; tumour size, shape irregularity (concavity and concave points) are the key signals of malignancy.
- Key findings: 15 feature pairs have correlation above 0.95; mean area follows an approximately Log-normal rather than Normal distribution; malignant counts in groups of 20 follow a Binomial distribution ($p = 0.3726$).
- Model performance: the tuned MLP achieved 96.5% accuracy, 96.3% benign recall and AUC = 0.990 on unseen data.
- Challenges: small data (569 records) makes metrics sensitive; correlated features make importance values unstable; the class imbalance and the unequal clinical cost of false negatives need care; no deep-learning library (TensorFlow/PyTorch) was required for a model of this size, so scikit-learn's MLP was used.

8.2 Limitations and future scope

- Use k-fold cross-validation and repeated runs to give confidence intervals for the metrics.
- Reduce redundancy with PCA or feature selection, and compare with baselines such as logistic regression, random forest and gradient boosting.
- Apply early stopping, dropout and class-weighting/threshold tuning to improve malignant recall.
- Use SHAP or LIME for patient-level explanations that doctors can trust.
- Extend to the raw FNA images using a CNN (transfer learning), and validate on external datasets from other hospitals before any clinical use.
- Disclaimer: this is an educational model and not a medical device.

9. References

5. W. H. Wolberg, W. N. Street and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: https://archive.ics.uci.edu/dataset/17
6. W. N. Street, W. H. Wolberg and O. L. Mangasarian, “Nuclear feature extraction for breast tumor diagnosis,” in Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology, vol. 1905, 1993, pp. 861–870.
7. F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
8. B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA: O'Reilly Media, 2007.
9. I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.
10. J. D. Hunter, “Matplotlib: A 2D graphics environment,” Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007.

10. Appendix – Code

The complete, commented source code is provided in the GitHub repository (eda_dl_project.py and the Jupyter notebook EDA_DL_Project.ipynb). Its main sections are:

- Imports and settings; data loading (Section 1–2 of the script)
- EDA: statistics, missing values, duplicates, IQR outliers
- Pre-processing: stratified split, winsorisation, standardisation
- Visualisation: 8 static charts + 1 interactive Plotly chart
- Model: MLP training loop, hyper-parameter grid search
- Evaluation: loss/accuracy curves, confusion matrix, ROC-AUC, permutation importance

Extract (first part of the script; see repository for the full code):

```
# %% [markdown]
```

```
# # EDA, Visualization and Deep Learning on the Breast Cancer Wisconsin (Diagnostic) Dataset
# Course: U21ADP05 – Exploratory Data Analysis and Visualization | Assignment II (Case Study)
#
# Pipeline: 1) Load  2) EDA  3) Preprocess  4) Visualize  5) Model (MLP)  6) Evaluate

# %% Imports and global settings
import json, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
from sklearn.datasets import load  breast  cancer
from sklearn.model  selection import train  test  split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, roc_curve,
                             roc_auc_score, accuracy_score, precision_score,
                             recall_score, f1_score, log_loss, classification_report)
from sklearn.inspection import permutation_importance
import plotly.express as px

warnings.filterwarnings("ignore")
SEED = 42
np.random.seed(SEED)
sns.set_theme(style="whitegrid", context="notebook")
FIG = "figures/"
R = {}                                  # results collected for the report

def save(name):
    plt.tight  layout(); plt.savefig(FIG + name, dpi=160, bbox  inches="tight"); plt.close()

# %% 1. DATA LOADING  (Stage 1-2 of visualization: ACQUIRE + PARSE)
raw = load_breast_cancer(as_frame=True)          # UCI Breast Cancer Wisconsin (Diagnostic)
df = raw.frame.copy()
# In sklearn, target 0 = malignant, 1 = benign. Re-label for readability.
df["diagnosis"] = df["target"].map({0: "Malignant", 1: "Benign"})
features = list(raw.feature_names)
print("Shape:", df.shape)
R["shape"] = list(df.shape); R["n_features"] = len(features)

# %% 2. EDA
print(df[features].describe().T.head(10))
R["missing_total"] = int(df.isna().sum().sum())
R["duplicates"] = int(df.duplicated().sum())
R["class_counts"] = df["diagnosis"].value_counts().to_dict()
R["dtypes"] = df[features].dtypes.astype(str).value_counts().to_dict()
desc = df[features].describe().T
desc["skew"] = df[features].skew()
desc.round(3).to_csv("outputs/summary_statistics.csv")
R["desc_sample"] = desc.loc[["mean radius","mean texture","mean area","mean concavity","worst perimeter"],
                            ["mean","std","min","50%","max","skew"]].round(3).reset_index().values.tolist()
print("Missing:", R["missing_total"], "Duplicates:", R["duplicates"])

# Outlier detection with the IQR rule
q1, q3 = df[features].quantile(.25), df[features].quantile(.75)
iqr = q3 - q1
out  mask = (df[features] < q1 - 1.5*iqr) | (df[features] > q3 + 1.5*iqr)
out_counts = out_mask.sum().sort_values(ascending=False)
R["outlier  top"] = out  counts.head(5).to  dict()
R["outlier_rows_any"] = int(out_mask.any(axis=1).sum())
```